

OS-Max: Maximizing Operating System Capabilities for High-Frequency Trading

Elia Zonta

May 29, 2026

Abstract

This report details the architectural design and system optimizations of OS-Max (HFT Signal Engine), an educational C++20 project aimed at exploring low-latency trading concepts. While acknowledging the limitations of standard operating systems compared to institutional kernel-bypass setups, this project implements advanced concurrency patterns, cache-aligned data structures, and hardware-level tuning to squeeze maximum performance from available hardware.

1 Introduction

High-Frequency Trading (HFT) requires deterministic execution, minimal latency, and zero-allocation critical paths. The OS-Max project demonstrates how to approach these requirements on a standard operating system (macOS/Linux) by applying strict memory semantics, thread pinning, and avoiding system calls during hot-path execution.

2 System Architecture

The engine operates on three dedicated threads, communicating via lock-free Single-Producer Single-Consumer (SPSC) ring buffers.

- **Feed Thread:** Ingests market data, updates the order book, and pushes raw ticks downstream.
- **Signal Thread:** Aggregates ticks into Time/Volume bars, evaluates technical indicators, and triggers trading signals.
- **Telemetry Thread:** Decoupled from the critical path, formatting and broadcasting ZeroMQ messages for real-time monitoring.

3 Low-Latency Techniques Implemented

3.1 Lock-Free Data Structures

The `SPSCQueue` uses `std::atomic` operations with strict `acquire` and `release` memory ordering. This allows data transfer between the Feed and Signal threads without the context-switching overhead of mutexes.

3.2 Cache Optimization and False Sharing Avoidance

Key structures (such as `Tick` and `OrderBook`) are explicitly aligned using `alignas(CACHE_LINE)`. In the `SPSCQueue`, the head and tail atomic pointers are padded into separate cache lines to prevent false sharing between producer and consumer cores.

3.3 Hardware Tuning

The engine provides a `run.sh` utility that applies Linux-specific optimizations when run as root:

- Disabling CPU C-states via `/dev/cpu_dma_latency` to prevent wake-up latency.
- Setting the CPU frequency governor to `performance`.
- Binding threads to specific NUMA nodes and logical cores (`taskset`, `numactl`).
- Rerouting IRQs away from the pinned engine cores.

4 Identified Areas for Improvement

Despite the robust architecture, several areas were identified for further enhancement:

1. **SIMD Mathematics Integrity:** The AVX2 implementation for Exponential Moving Averages (EMA) incorrectly vectorizes across the time series instead of computing sequentially.
2. **Process Lifecycle Management:** The engine lacks POSIX signal handlers (e.g., `SIGINT`) and spins indefinitely in the main thread, resulting in abrupt termination and preventing graceful resource cleanup.
3. **Queue Spin-locking:** Over-saturation of queues can lead to indefinite spin-locks; implementing a yield or ring-buffer overwrite mechanism for telemetry would improve resilience.

5 Conclusion

OS-Max successfully showcases a hobbyist implementation of institutional HFT concepts. While limited by the POSIX networking stack and non-real-time kernel scheduling of macOS/standard Linux, the application of cache-aware C++20 code and lock-free concurrency achieves commendable latency optimization.